Improving semantic movie search

Experiments, implementations and measured outcomes

Idea → implementation → comparison → verdict

Master 2 project

July 2026

How every experiment is presented

Each experiment follows the same three questions:

1. Setup and idea: what problem were we trying to solve?
2. Implementation: what changed in code or prompts? A concrete example and source citation are provided.
3. Comparison and verdict: did it beat the previous method, and why?

Quality measures

Recall = share of correct movies recovered. Precision = share of returned movies that are correct. F1 = balance between precision and recall.

Starting point: two main methods

SUQL Stage 2

- Cheap model scores each review.
- Strong model checks uncertain cases.
- Designed to save strong-model calls.

V3.2

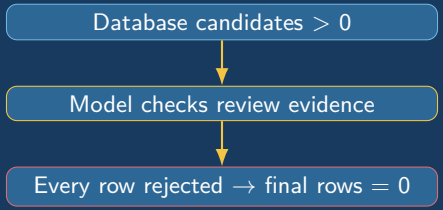
- Database rules reduce the candidate set.
- Reviews are classified in batches.
- Usually uses only about four model calls per run.

These are the reference methods against which prompt, context, cascade and synonym changes were tested.

Evidence: `project SUQL/Stage_2/README.md:14-55; project Trummer/heterogen_v3_2/`

Experiment 1 — setup: recover missing final rows

Observation: the database found candidates, but some Gemma4/SUQL runs returned zero movies.



Idea: uncertainty or a scoring failure should trigger a second check, not automatic deletion.

Evidence: `test_project/project SUQL/src_baseline_stage2/suql_engine.py:706-717`

Experiment 1 — implementation: safer confidence handling

Before

Prompt

```
Classify whether the review answers
the question with Yes.
Return exactly one token:  1 or 0.
```

- Expected confidence for both tokens.
- A confident negative could be rejected immediately.

After

Prompt

```
Avoid false negatives.  If evidence is
credible, mixed or ambiguous, answer 1.
Answer 0 only when clearly absent.
```

- Can derive signed confidence from one generated-token probability.
- Missing confidence goes to the stronger model.
- Negative cheap scores are rechecked.

Evidence: `test_project/project SUQL/Stage_1/profiler.py:54-60,136-147; Stage_2/cascade_filter.py:269-308`

Experiment 1 — comparison and verdict

Before fix

- Some 12B→31B SUQL configurations: recall = 0.000.
- More output tokens still produced zero rows.

After recall-first redesign

- Qwen Stage 2 baseline: recall = 0.692.
- Final rows were produced again.
- Cost increased to 33.5 calls/run.

Verdict: succeeded

The problem was an architecture–model compatibility failure. Models exposed the weakness, but the architectural fix was to recheck uncertainty instead of rejecting it.

Evidence: `test_project/results/baseline_10q_10reps/qwen_comparison/aggregate_summary.csv`

Experiment 2 — setup and implementation: extra examples

Idea: give the model labelled examples of recommendation, chemistry and ending evidence before it judges a new review.

Example added to the prompt

```
Ground-truth examples for this exact task:
Example 1 review:  [known review]
Example 1 label:  Yes
Apply the same distinction to the new review.
```

The code selected an example group from predicate words, formatted the reviews and labels, and prepended them to the classifier prompt.

Evidence: test_project/context_version/fewshot_context.py:9–24; project SUQL/Stage_2/cascade_filter.py

Experiment 2 — comparison and verdict

Method	Previous recall	With examples	Change
Stage 2	0.556	0.194	−0.361
V3.2	0.611	0.556	−0.056

Verdict: failed

The extra reviews made prompts longer and introduced distracting evidence. The model became slower and more conservative instead of transferring the intended distinction.

Evidence: `test_project/context_version/results/3q_context_qwen35_1rep/comparison_averages.csv`

Experiment 3 — setup: expert synonym guidance

Idea: provide a short list of human-selected ways to express the same concept.

Example for romantic chemistry

```
Semantic family:  relationship_and_chemistry_praise.
Look for these signals and natural paraphrases:
chemistry, convincing couple, believable relationship, romance works.
The terms are recall cues, not exact-match requirements.
```

The guide was attached to the semantic predicate before model classification.

Evidence: `test_project/generalized_synonym_version/predicate_synonyms.py:48-59`

Experiment 3 — comparison and verdict

Method	Previous recall	Expert cues	Change
Stage 2	0.556	0.583	+0.028
V3.2	0.611	0.667	+0.056

Verdict: small success

Short, question-specific paraphrases helped the model recognize alternative wording. The gain was modest because broader acceptance also reduced precision.

Evidence: `test_project/synonym_version/results/3q_dynamic_synonyms_qwen35/comparison_averages.csv`

Experiment 4 — setup and implementation: learned cues

Idea: learn vocabulary automatically instead of writing every synonym by hand.

Lexical learner

- Selects words/phrases correlated with positive examples.
- Risk: frequent artifacts look predictive.

Ground-truth concept learner

- Uses known positive movies to organize cues into semantic topics.
- Aims to learn meanings rather than isolated tokens.

Both produced a JSON guide mapping each question to a semantic family and cue categories.

Evidence: `test_project/learned_synonym_version/learn_synonym_guide.py`; `ground_truth_synonym_version/learn_synonym_guide.py`

Experiment 4 — comparison and verdict

Variant	Method	Recall	Change vs baseline
Lexical learning	Stage 2	0.361	−0.194
Lexical learning	V3.2	0.306	−0.306
GT concepts	Stage 2	0.361	−0.194
GT concepts	V3.2	0.583	−0.028

Verdict: lexical failed; concept learning was partial

Lexical selection learned accidental correlations and dataset artifacts. Semantic supervision repaired much of the V3.2 damage, but did not consistently beat expert cues.

Evidence: `test_project/*synonym_version/results/3q_*/comparison_averages.csv`

Experiment 5 — setup: learn from a larger 5k sample

Idea: improve coverage by learning concepts from 5,000 IMDb rows while preventing benchmark leakage.

- Excluded all 600 benchmark movies.
- Split remaining movies into training and validation sets by movie.
- Started from high-precision seed cues.
- Added phrases only when they improved validation recall above a precision threshold.

Example: recommendation learning used 688 pseudo-positive movies and selected 9 cues.

Evidence: `test_project/generalized_synonym_version/learn_full_dataset_synonyms.py`; `full5k_concept_learning_report.json`

Experiment 5 — implementation: what the model actually sees

Learned recommendation cues

Guide entry

```
paraphrases:  must see, worth watching, worth seeing,
recommend seeing, recommend watching
implicit signals:  worth a watch, end i recommend,
recommend that you see
```

Prompt prefix sent to the model

Rendered guidance

```
Learned semantic concept:  explicit_recommendation.
Paraphrases:  must see, worth watching, ...
Implicit signals:  worth a watch, ...
These are semantic examples, never exact-match rules.  Accept unseen wording.
```

This prefix is concatenated before the review and question in both Stage 2 and V3.2.

Evidence: predicate_synonyms.py:43-76; project SUQL/Stage_2/cascade_filter.py:343-353; heterogen_v3_2/trummer_join/cascade.py:546-574

Experiment 5 — comparison and verdict

Variant	Method	Precision	Recall	F1
No guide	Stage 2	.652	.491	.520
Small guide	Stage 2	.732	.591	.610
Full-5k guide	Stage 2	.743	.528	.576
No guide	V3.2	.692	.481	.546
Small guide	V3.2	.743	.556	.617
Full-5k guide	V3.2	.733	.583	.636

Verdict: succeeded for V3.2, partial for Stage 2

More data improved V3.2 recall and F1. It did not beat the smaller guide for Stage-2 recall: frequent phrases can displace rare, question-specific cues. Matched Q1–Q9 only; Q10 is incomplete.

Evidence: `test_project/results/gemma4_baseline_vs_small_vs_full5k_synonyms/matched_aggregate.csv`

Experiment 6 — setup and implementation: three model levels

Idea: add a middle model to recover hard candidates before final adjudication.

1. Qwen 4B: fast pass for explicit evidence.
2. Granite 8B: recovery pass over rejected rows; searches for paraphrases and indirect evidence.
3. Qwen 9B: final decision over surviving candidates, or all candidates when recovery is short.

Each model returned structured JSON decisions for every candidate.

Evidence: `test_project/three_level_version/three_level_pipeline.py:55-104,107-128`

Experiment 6 — comparison and verdict

Method	Two-level recall	Three-level recall	Change
Stage 2	.556	.139	−.417
V3.2	.611	.083	−.528

Verdict: failed

Every additional model boundary created another opportunity to reject a correct movie. Errors accumulated, while runtime rose to roughly 300 seconds.

Evidence: `test_project/three_level_version/results/3q_three_level_qwen_granite_qwen/comparison_averages.csv`

Overall conclusions by method

Method	Verdict	Simple reason
Recall-first confidence fix	Success	Prevented uncertainty from silently deleting candidates.
Extra labelled context	Failed	Added prompt noise and conservative decisions.
Expert synonyms	Small gain	Short cues matched the questions well.
Lexical learning	Failed	Learned correlations rather than stable meaning.
GT concept learning	Partial	Better semantics, but inconsistent generalization.
Full-5k learning	Partial / useful	Best V3.2 quality; smaller guide kept better Stage-2 recall.
Three-level cascade	Failed	More rejection points compounded errors.
V3.2	Best balance	Strong quality with about four calls per run.

Final recommendation

1. Use V3.2 + full-5k guide for the best current balance of precision, recall, F1 and calls.
2. Use Stage 2 + small guide when recall is the priority and runtime is less important.
3. Keep the recall-first fallback rule in every method: scoring failure must never equal rejection.
4. Keep prompts short and task-specific; do not add context or model stages without proving a recall gain.
5. Finish Q10 and validate on unseen movies before making a full ten-question generalization claim.